



viettel

Theo cách của bạn



VIETTEL DIGITAL TALENT 2026

Streaming Lakehouse Data Platform

Learning how a modern streaming platform works, end to end

Mentor: Nguyen Minh Hieu

Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1st, 2026

Table of Contents

- 1 Introduction
- 2 Background & Theory
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Conclusion
- 6 References

Why a streaming lakehouse

- A music-streaming platform produces a continuous stream of small events, not daily batches
- Batch ETL is always some interval behind reality – fine for reports, too slow for "what is happening now"
- Doing this reliably (not just fast) means solving ordering, fault tolerance, exactly-once delivery, and read/write contention

Goals

- Build a working, end-to-end streaming lakehouse pipeline and learn how a modern streaming platform works
- Apply distributed-systems concepts from the course to a real system: partitioning, replication, exactly-once processing, lag monitoring
- Make the whole system observable and reproducible – single Docker Compose command

Message queue (pub-sub)

- Producers append to a durable, ordered log; consumers subscribe independently, at their own pace
- Decouples producers from consumers in both time and number

Batch vs. streaming engine

- Batch: bounded input, runs once, stops
- Streaming: unbounded input, engine never stops waiting for the next event (micro-batch or event-at-a-time)

Storage & open table formats

- Lakehouse = object storage (cheap, scalable) + an open table format on top for ACID upserts and time travel
- COPY_ON_WRITE – fast reads, expensive writes; MERGE_ON_READ – cheap streaming appends, merged at read time

Medallion layering

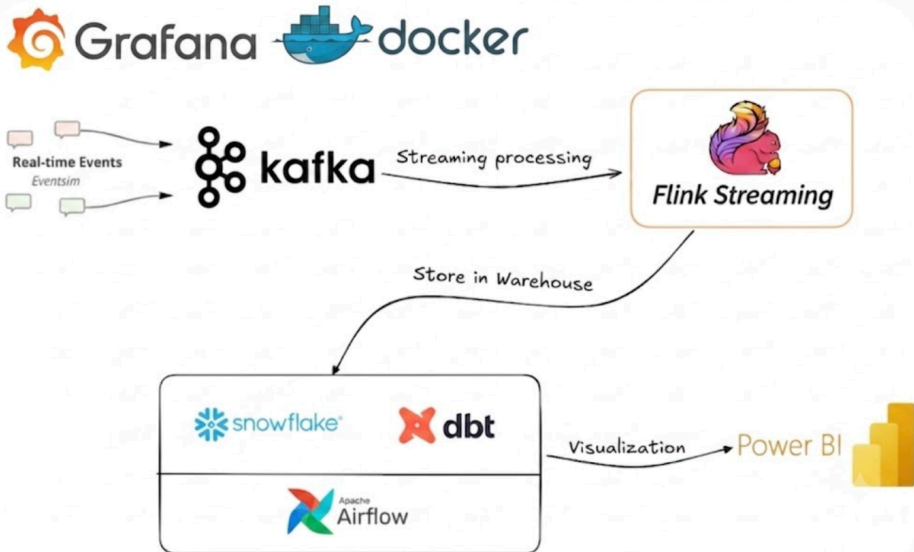
- Bronze (raw) → silver (cleaned) → gold (modeled)
- This project: bronze → gold directly – dataset is small and clean enough to skip silver

The Problem Before Streaming Lakehouses

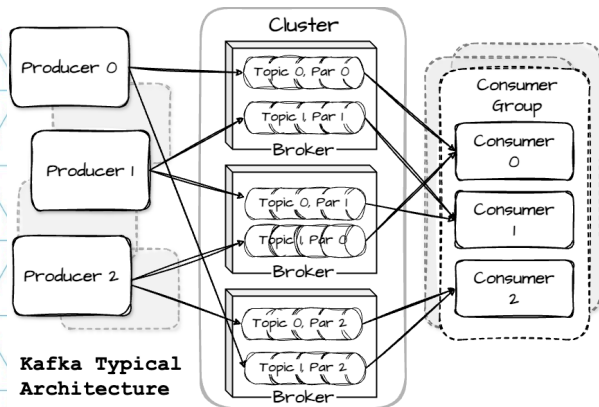
- **Batch ETL latency** – dashboards always reflect the last scheduled load, not the current state
- **Lambda-architecture complexity** – a batch layer and a separate real-time speed layer duplicate the same logic in two codebases
- **No serving layer on the lake** – plain object storage has no row-level update/delete or ACID transactions; the lake stayed a read-only dump
- **Storage and compute coupled** – traditional warehouses force paying for both together, so teams kept two copies of the same data

This project's concrete challenges: keeping up with Eventsim's continuous arrival rate across 3 Kafka topics, exactly-once fault-tolerant processing on a 2-node cluster, and fitting everything inside a fixed, shared 4-core Spark budget.

Streamify: End-to-End Architecture

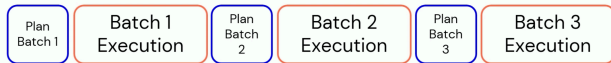


Ingestion Layer: Apache Kafka



- 2-node KRaft cluster (broker-1, broker-2) – no ZooKeeper, each node is broker & controller
- 3 topics (listen_events, page_view_events, auth_events), 3 partitions each, replication factor 2
- Every partition has a leader on one broker and a follower on the other – follower takes over if its broker fails

Stream Processing: Spark Structured Streaming → Hudi Bronze



Driver polls Kafka, groups new records into a micro-batch, runs it as a Spark job

- Reads all 3 topics, fixes latin1-mojibake, derives time columns, appends to staging.<topic> (Hudi MERGE_ON_READ)
- Checkpoint volume tracks Kafka offsets and query state – resumes exactly where it left off on restart
- Shares the 4-core Spark Standalone cluster with the dbt Thrift Server, 2 cores each

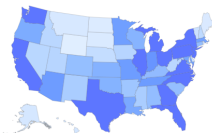
Transformation & Serving



- dbt builds a **star schema** (fact_streams + 5 dims) and a denormalized **wide_streams** table straight from bronze, via the Spark Thrift Server
- Both materialized as COPY_ON_WRITE Hudi tables, rebuilt each run
- **Superset** queries wide_streams through the same Thrift Server – one flat table, no dashboard-time joins

STREAMIFY

User Activity per State



Gender Distribution

● M ● F

User Level By Gender

● free ● paid

User Activity

10K 4K

State Filter
All

Streams

327K

Total Users

16.39K

Date Range

8/20/2025 8/27/2025

Chart Busters

Song

Streams

Yo Yo Man (Garter Belt)	1,352
Evenglow	812
Up Jumped The Devil - Live	778
Una dolce melodia	754
Close Your Eyes	753
Wish (Album Version)	642
AIM, Ella No Cambia Nada	588
Intro	447
So Damn High (Will Eastman Club Edit)	398
Inferior (Late Night Version)	312

Top Artists

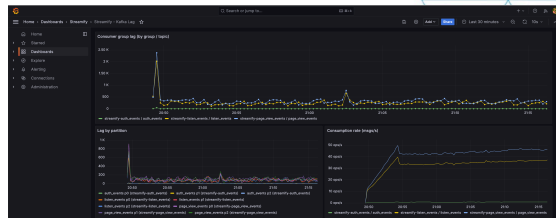
Artist

Streams

Tony Joe White	1,585
David & Steve Gordon	906
Atomic Rooster	861
Clarence Gatemouth Brown	815
Franco, Valeriana	754
Luis Alberto Spinetta	699
Beautiful Creatures	642
Phil Collins	556
Sugar Minott	521
The Jackson Southernaires	485

Monitoring Layer

- **kafka-exporter** – per-partition consumer-group lag
- **Spark's Prometheus endpoint** – job/JVM metrics
- **cAdvisor** – per-container CPU/memory
- Grafana renders lag and Spark job health so the system's health is visible, not just whether it is running



Conclusion

Results Achieved

- Understood, hands-on, how a streaming platform works end to end: why a Kafka partition needs a follower replica, why the Structured Streaming checkpoint is separate from the consumer-group offset, when MERGE_ON_READ is worth its read-time cost
- Exercised partitioning, replication, checkpointed exactly-once processing, and lag observability against real failure modes, not just in theory
- Confirmed why the streaming-lakehouse pattern closes the batch/lambda-architecture gaps: no duplicated speed-layer logic, a transactionally-updatable lake, storage and compute that scale independently

Limitations

- Small, fixed 4-core Spark cluster, no autoscaling
- Synthetic Eventsim dataset, not validated against real production traffic

Future Directions

- Real or real-scale streaming dataset; autoscaled Kafka/Spark cluster
- Alerting on consumer lag and job failures; a silver layer once bronze needs cleaning

References

- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- Apache Hudi Documentation. <https://hudi.apache.org/docs/overview>
- dbt Documentation (dbt-spark adapter).
<https://docs.getdbt.com/docs/core/connect-data-platform/spark-setup>
- Apache Superset Documentation. <https://superset.apache.org/docs/intro>
- Eventsim (viirya's fork). <https://github.com/viirya/eventsim>
- Million Song Dataset. <http://millionsongdataset.com>

- THE END -

Thank you for your attention

Contact:

Ngo Quang Hieu — Viettel Digital Talent 2026